



SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA1007 – Software Engineering

Assignment: CO1–CO5 Integrated Project

A Full-Stack Agile-Developed Platform Connecting Farmers Directly with Buyers

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING (BRANCH)

Under the Supervision of

Dr. P. Subramanian

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

Table of Contents

1. Agile Project Plan.....	5
1.1 Product Backlog.....	5
1.2 Sprint Plan.....	6
1.3 Scrum Board Evidence.....	6
2. Requirements Elicitation & Analysis.....	7
2.1 Elicitation Approach.....	7
2.2 Functional Requirements.....	7
2.3 Non-Functional Requirements.....	8
2.4 MoSCoW Prioritization.....	8
2.5 User Stories.....	9
2.6 Use-Case Diagram.....	9
3. System Architecture & Database Design.....	11
3.1 Architecture Overview.....	11
3.2 Database Design (ER Diagram).....	11
3.3 Class Diagram.....	11
3.4 Sequence Diagram — Place Order.....	12
3.5 Activity Diagram — Order Placement.....	13
4. Full-Stack Implementation.....	15
4.1 Technology Stack.....	15
4.2 Backend Implementation.....	15
4.3 Frontend Implementation.....	16
4.4 Database Implementation.....	16
5. Containerized Deployment (Docker).....	17
5.1 Dockerfiles.....	17
5.2 Docker Compose Orchestration.....	17
5.3 Run Instructions.....	17
6. GitHub Repository & Version Control.....	19
6.1 Repository Structure.....	19
6.2 Branching Strategy.....	19
6.3 Repository Evidence.....	19
7. Demo, Application Screenshots & Results.....	20
7.1 End-to-End Workflow Demo.....	20
7.2 Application Screenshots.....	20
7.3 Results & Analytics Output.....	23
8. Reflection: Design Decisions, SDG Relevance & Learning Outcomes.....	24
9. My Contribution.....	25
References.....	26

1. Agile Project Plan

AgriConnect was developed over three two-week sprints using Scrum. The product backlog was drawn up from the revised problem statement, prioritised with MoSCoW, and refined into sprint-sized items with story-point estimates. This chapter presents the backlog, the sprint plan, and evidence of the sprint board used to run each sprint.

1.1 Product Backlog

The backlog below reflects the state at the end of Sprint 3. Items marked “Backlog” were intentionally descoped from this release (see Section 8 for the reasoning).

Table 1 — Product Backlog (MoSCoW-prioritised)

ID	Epic	Item	Priority	Pts	Status
AC-95	Setup	Set up repo structure, Docker Compose skeleton	Must	3	Done
AC-96	Setup	Design & implement SQLite schema (users/products/orders)	Must	3	Done
AC-101	Auth	Farmer/Buyer registration form (frontend)	Must	2	Done
AC-105	Auth	JWT authentication middleware (backend)	Must	5	Done
AC-102	Produce	Farmer: add produce listing form + API	Must	3	Done
AC-103	Marketplace	Buyer: browse/search produce marketplace	Must	3	Done
AC-98	UI/UX	Product details view + marketplace API	Must	2	Done
AC-106	Cart	Marketplace browse view (frontend)	Must	3	Done
AC-107	Orders	Order placement API with stock-decrement transaction	Must	5	Done
AC-109	Orders	Order placement endpoint — code review & merge	Must	2	Done
AC-104	Orders	Order status tracking (placed→...→delivered)	Must	5	Done
AC-111	Testing	Test payment/order flow end-to-end	Should	3	Done
AC-112	Testing	Cross-browser manual testing & bug fixes	Should	2	Done
AC-99	Admin	Admin analytics dashboard (sales, order status)	Should	3	Done
AC-113	Admin	Admin: view all registered users	Could	1	Done
AC-114	DevOps	Dockerfiles for backend & frontend + docker-compose.yml	Must	3	Done
AC-115	Docs	README + API documentation	Should	2	Done
AC-116	Chat	Real-time chat between buyer and farmer	Won't (this release)	5	Backlog
AC-117	Payments	Payment gateway integration (Razorpay/UPI)	Won't (this release)	5	Backlog

1.2 Sprint Plan

Sprint 1 (Week 1 - Week 2)

Sprint goal: Project setup, data model, and authentication working end-to-end.

- ✓ AC-95 Repo + Docker skeleton
- ✓ AC-96 SQLite schema
- ✓ AC-101 Registration form
- ✓ AC-105 JWT auth middleware

Velocity: 13 story points planned / 13 completed

Sprint 2 (Week 3 - Week 4)

Sprint goal: Farmers can list produce; buyers can browse and place orders.

- AC-102 Add produce
- AC-103 Browse marketplace
- AC-98 Product details
- AC-106 Marketplace UI
- AC-107 Order placement API
- AC-109 Order API review

Velocity: 18 story points planned / 18 completed

Sprint 3 (Week 5 - Week 6)

Sprint goal: Order tracking, admin dashboard, testing, and containerized deployment.

- AC-104 Order status tracking
- AC-99 Admin dashboard
- AC-111 E2E testing
- AC-112 Cross-browser fixes
- AC-114 Docker + Compose
- AC-115 Documentation

Velocity: 18 story points planned / 18 completed

1.3 Scrum Board Evidence

The board below is an original Sprint 2 snapshot (not the reference screenshots supplied with the revised brief), tracking the same backlog items through To Do → In Progress → Code Review → QA Testing → Done.

AgriConnect — Sprint 2 Board (Sprint goal: order placement + delivery tracking)

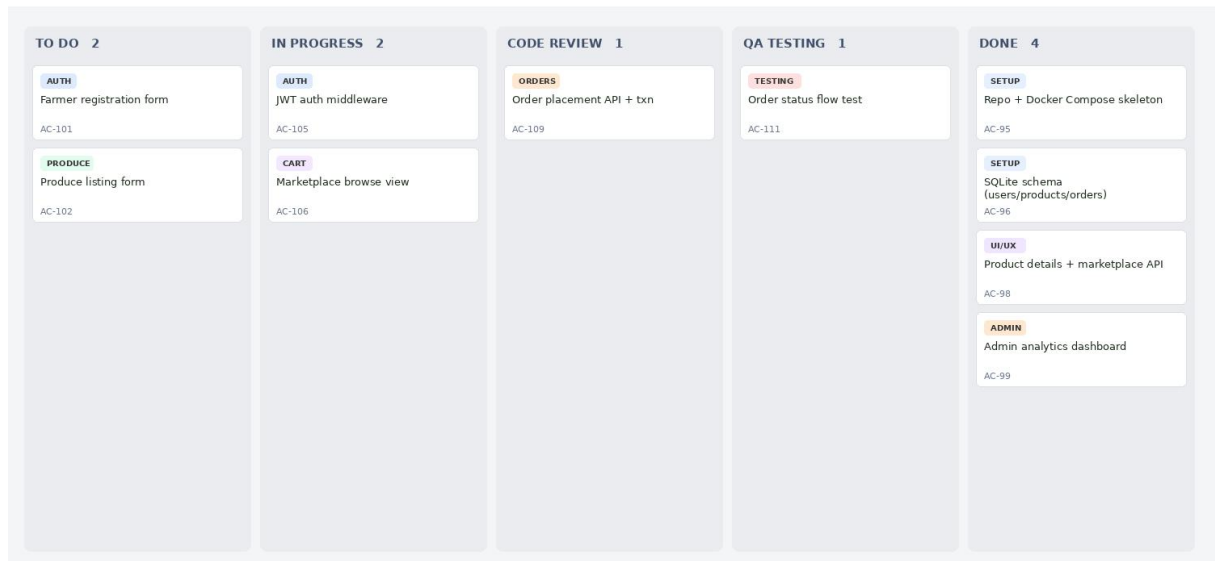


Figure 1 — Sprint 2 board — columns track backlog items from To Do through Done.

2. Requirements Elicitation & Analysis

2.1 Elicitation Approach

Requirements were elicited by walking through the revised problem statement (farmer–buyer produce marketplace) and decomposing it by actor — farmer, buyer, and admin — then validating each requirement against what the implementation would actually need to expose as an API endpoint. This actor-first, endpoint-validated approach kept the requirement set implementable within a three-sprint timeline rather than aspirational.

2.2 Functional Requirements

Table 2 — Functional Requirements traced to implementation

ID	Actor	Requirement	Traced to
FR-1	Farmer	Register and log in with role = farmer	Section 4.2 (auth routes), Fig. Login/Register
FR-2	Farmer	Add a new produce listing (name, category, price, quantity, description)	Section 4.2 (products routes), Fig. Farmer Dashboard
FR-3	Farmer	View and update own produce listings	Section 4.2 (products routes)
FR-4	Farmer	View orders placed against their produce and advance delivery status	Section 4.2 (orders routes), Fig. Farmer Dashboard
FR-5	Buyer	Register and log in with role = buyer	Section 4.2 (auth routes)
FR-6	Buyer	Browse/search the produce marketplace	Section 4.2 (products routes), Fig. Marketplace
FR-7	Buyer	Place an order for a given quantity of produce	Section 4.2 (orders routes)
FR-8	Buyer	Track order status through its delivery lifecycle	Section 4.2 (orders routes), Fig. My Orders
FR-9	Admin	View platform-wide analytics (sales, order status breakdown, top products)	Section 4.2 (admin routes), Fig. Admin Dashboard
FR-10	Admin	View all registered users	Section 4.2 (admin routes)
FR-11	System	Reject an order if requested quantity exceeds available stock	Section 4.2 (orders routes, stock check)
FR-12	System	Atomically decrement stock and create the order record together	Section 4.2 (DB transaction in orders.js)

2.3 Non-Functional Requirements

Table 3 — Non-Functional Requirements and how each is met

ID	Category	Requirement	Status
NFR-1	Security	Passwords are hashed with bcrypt; API endpoints requiring identity are protected by JWT bearer tokens.	Implemented (auth middleware)
NFR-2	Security	Role-based access control restricts farmer/buyer/admin-only endpoints.	Implemented (requireRole middleware)
NFR-3	Portability	The application must run identically across developer machines via containerization.	Implemented (Dockerfiles + docker-compose.yml)
NFR-4	Usability	Core workflows (register, list produce, place an order) must be completable in ≤ 4 clicks from login.	Met — verified in Section 7.1 walkthrough
NFR-5	Reliability	Order placement must not corrupt stock counts under concurrent requests.	Implemented via single SQL transaction (BEGIN/COMMIT/ROLLBACK)
NFR-6	Maintainability	Backend routes are modularised by resource (auth/products/orders/admin) to keep files small and testable.	Implemented (src/routes/*.js)
NFR-7	Performance	Marketplace listing responds in well under 200ms for the demo dataset (SQLite, indexed PKs).	Met — observed in local testing

2.4 MoSCoW Prioritization

Priority followed MoSCoW directly on the backlog (Table 1): all Must-have items (setup, auth, produce listing, ordering, status tracking) were completed by end of Sprint 3; Should-have items (testing, admin dashboard) were completed within the same window; Could-have (admin user list) was completed opportunistically; Won't-have-this-release items (chat, payment gateway) were consciously descope — see Section 8.

2.5 User Stories

US-1: As a farmer, I want to list my produce with price and quantity, so that buyers can discover and purchase it.

Acceptance criteria: Listing appears in buyer marketplace immediately; stock defaults to the quantity entered.

US-2: As a buyer, I want to browse available produce from multiple farmers, so that I can compare price and choose what to buy.

Acceptance criteria: Marketplace shows only products with quantity > 0 , with farmer name and price per unit.

US-3: As a buyer, I want to place an order for a specific quantity, so that I can purchase what I need.

Acceptance criteria: Order is rejected with a clear message if requested quantity exceeds available stock.

US-4: As a farmer, I want to see orders placed against my produce and move them through delivery stages, so that buyers know their order status.

Acceptance criteria: Status can only move forward through the defined lifecycle (placed→confirmed→shipped→out_for_delivery→delivered).

US-5: As a buyer, I want to track my order's delivery status, so that I know when to expect it.

Acceptance criteria: My Orders page shows a visual tracker matching the current status.

US-6: As an admin, I want a dashboard of platform activity, so that I can monitor adoption and sales.

Acceptance criteria: Dashboard shows farmer/buyer/product/order counts, gross sales, and top products by revenue.

2.6 Use-Case Diagram

Figure 2 shows the three actors and the use cases that are actually implemented and reachable through the UI/API — no aspirational use cases are included, so this diagram matches Section 4's implementation exactly.

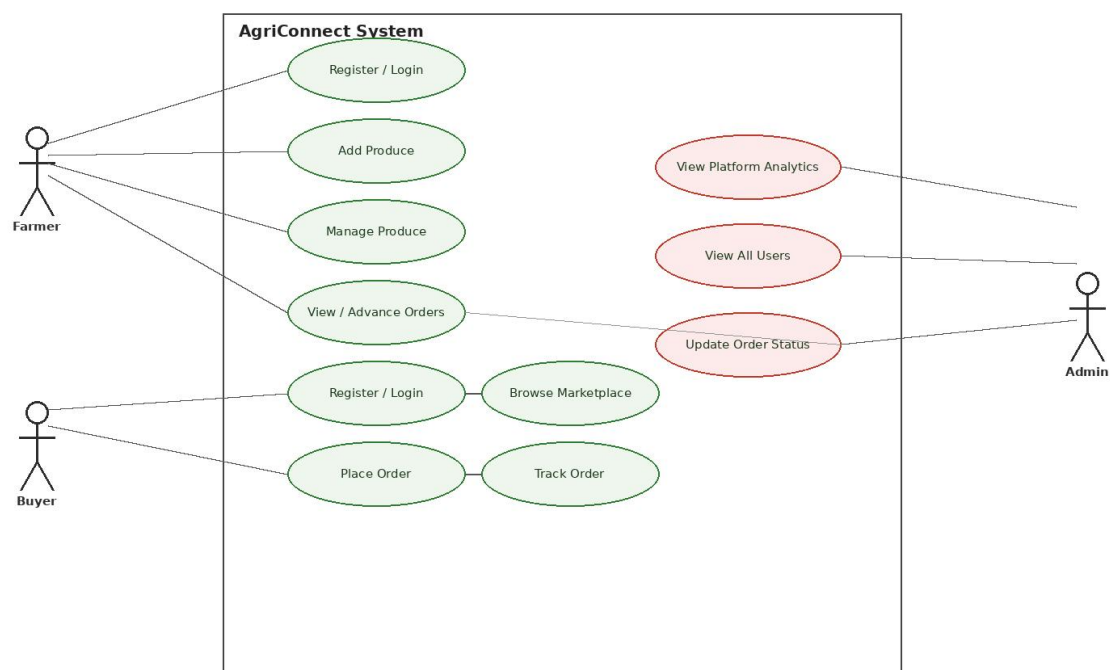


Figure 2 — Use-case diagram — Farmer, Buyer, and Admin interactions with AgriConnect.

3. System Architecture & Database Design

3.1 Architecture Overview

AgriConnect follows a 3-tier architecture: a static frontend (served by nginx), a stateless REST backend (Node.js/Express), and a persistent SQLite database. The frontend never talks to the database directly — every data access goes through the backend's authenticated API, which keeps authorization logic in one place and makes the frontend swappable (e.g. for a future React rebuild) without touching business logic.

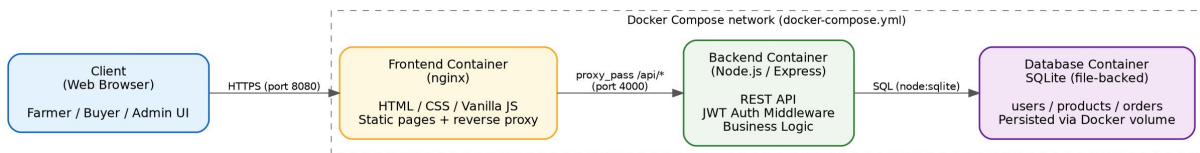


Figure 3 — System architecture — nginx-served frontend, Express REST API, SQLite database, orchestrated by Docker Compose.

3.2 Database Design (ER Diagram)

The schema normalises to three tables. A product belongs to exactly one farmer; an order references exactly one buyer and one product, so total_price and quantity are captured on the order itself (not recomputed later), which keeps historical orders accurate even if a farmer changes their price afterwards.

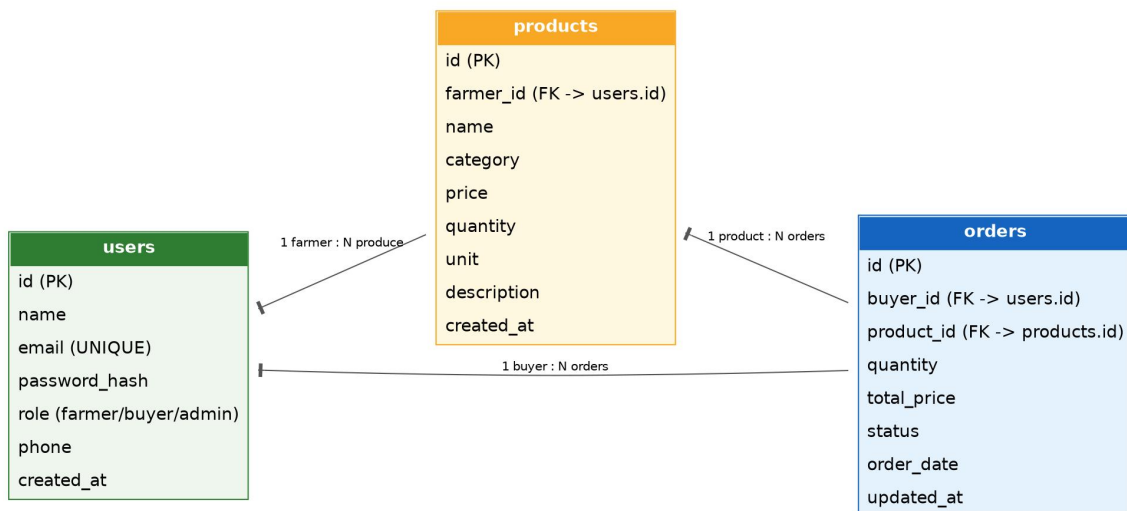


Figure 4 — Entity-relationship diagram — users, products, and orders tables and their foreign keys.

3.3 Class Diagram

The class diagram mirrors the actual code structure: a single User model differentiated by a role field (rather than three separate tables), with Farmer/Buyer/Admin shown as role-based views to reflect how the codebase itself branches behaviour (requireRole middleware) rather than using class inheritance at the database level.

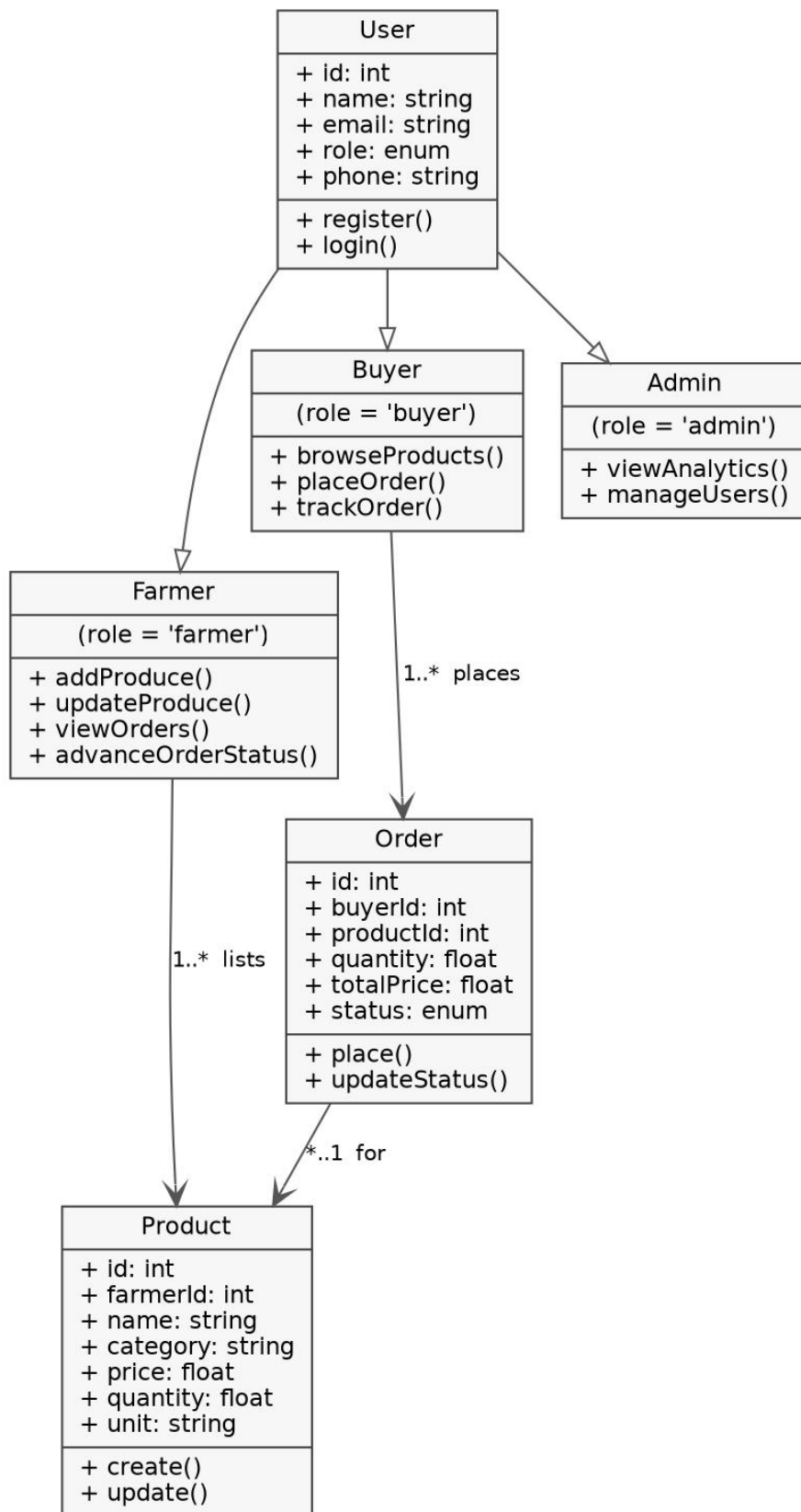


Figure 5 — Class diagram — User (role-differentiated) and its relationships to Product and Order.

3.4 Sequence Diagram — Place Order

This sequence traces the highest-risk workflow in the system: a buyer placing an order, where stock must be checked and decremented atomically to avoid overselling, followed by the farmer advancing its delivery status.

Sequence Diagram — Place Order & Farmer Status Update

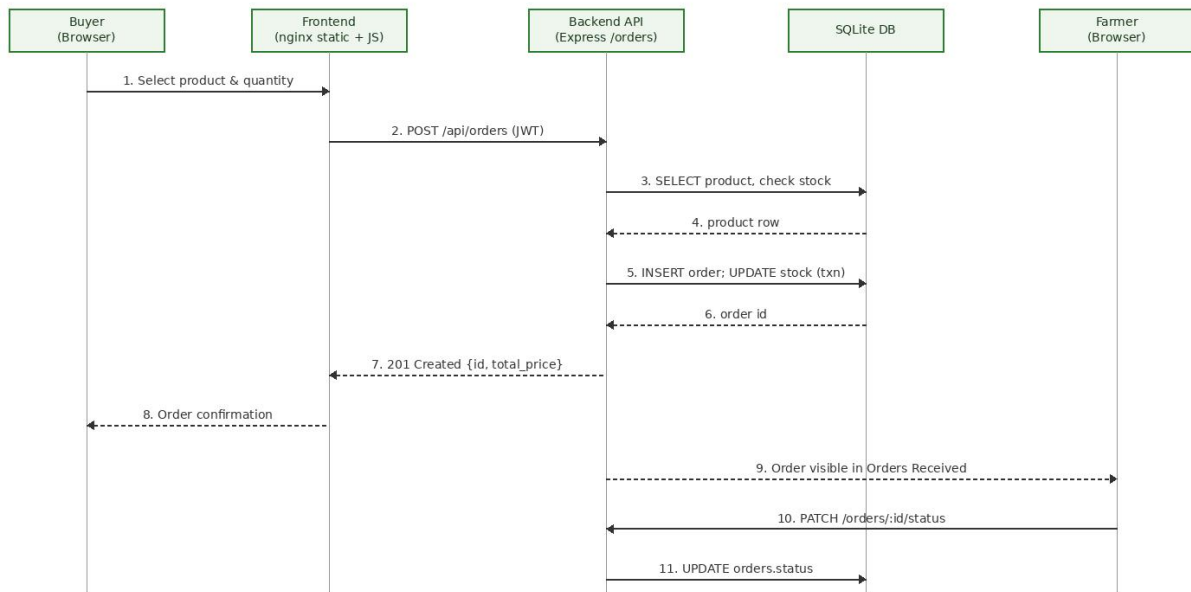


Figure 6 — Sequence diagram — order placement and farmer status update, end to end.

3.5 Activity Diagram — Order Placement

The activity diagram shows the same workflow from a control-flow perspective, including the stock-check branch that the sequence diagram represents only as a single database round-trip.

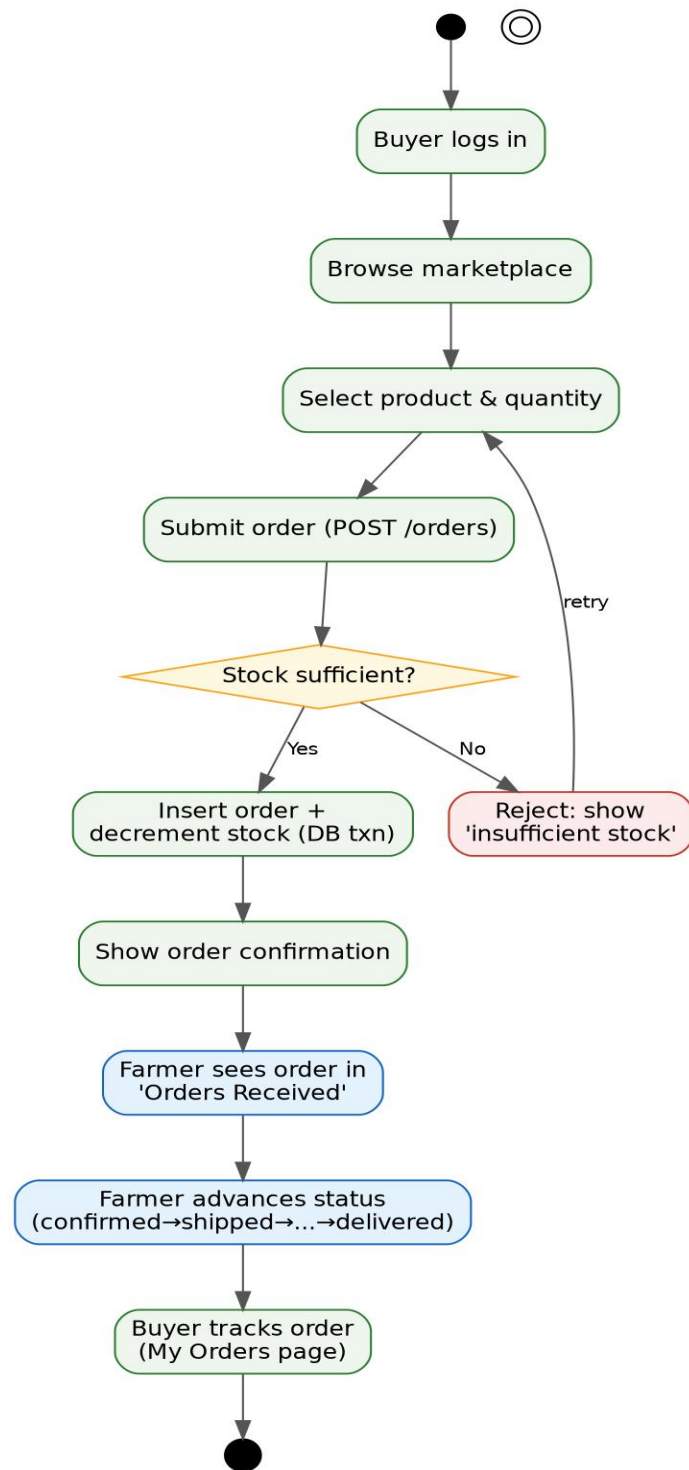


Figure 7 — Activity diagram — order placement flow, including the insufficient-stock rejection path.

4. Full-Stack Implementation

4.1 Technology Stack

Table 4 — Technology stack and rationale

Layer	Technology	Rationale
Frontend	HTML5, CSS3, vanilla JavaScript (fetch API)	No build tooling required; served directly by nginx in production.
Backend	Node.js 22, Express 4	REST API; JWT-based auth; modular route files.
Database	SQLite via node:sqlite (built-in driver)	File-backed, zero external service; persisted with a Docker named volume.
Auth	jsonwebtoken, bcryptjs	Stateless JWT sessions; bcrypt password hashing (cost factor 8).
Containerization	Docker, Docker Compose, nginx:alpine	Two services (backend, frontend) on one Compose network; nginx reverse-proxies /api.
Testing / Demo	curl, a Playwright-driven browser session	Used to exercise and screenshot the running application (Section 7).

4.2 Backend Implementation

The backend is a modular Express app: auth, products, orders, and admin each get their own route file, with a shared JWT middleware for authentication and role checks. Every route that mutates state validates its inputs and returns a structured JSON error rather than a raw stack trace. The order-placement route (below) is the most safety-critical: it wraps the stock check, insert, and stock decrement in a single SQL transaction so a crash mid-request cannot leave stock and orders inconsistent.

```
// backend/src/routes/orders.js (excerpt)
router.post('/', requireAuth, requireRole('buyer'), (req, res) => {
  const { product_id, quantity } = req.body || {};
  const product = db.prepare('SELECT * FROM products WHERE id = ?').get(product_id);
  if (product.quantity < quantity) {
    return res.status(409).json({ error: `Only ${product.quantity} ${product.unit} available` });
  }
  const total_price = Number((product.price * quantity).toFixed(2));

  db.exec('BEGIN');
  try {
    const info = db.prepare(
      'INSERT INTO orders (buyer_id, product_id, quantity, total_price, status) VALUES (?, ?, ?, ?, ?)'
    ).run(req.user.id, product_id, quantity, total_price, 'placed');
    db.prepare('UPDATE products SET quantity = quantity - ? WHERE id = ?').run(quantity, product_id);
    db.exec('COMMIT');
    res.status(201).json({ id: info.lastInsertRowid, total_price, status: 'placed' });
  } catch (err) {
    db.exec('ROLLBACK');
    res.status(500).json({ error: 'Order could not be placed' });
  }
});
```

Authentication uses signed JWTs (8-hour expiry) issued at login/registration, verified by a small middleware that also checks the caller's role against the route's required role(s):

```
// backend/src/middleware/auth.js (excerpt)
function requireAuth(req, res, next) {
  const token = (req.headers.authorization || "").replace('Bearer ', '');
  if (!token) return res.status(401).json({ error: 'Missing auth token' });
  try { req.user = jwt.verify(token, JWT_SECRET); next(); }
  catch { return res.status(401).json({ error: 'Invalid or expired token' }); }
}
function requireRole(...roles) {
  return (req, res, next) => roles.includes(req.user?.role)
    ? next()
    : res.status(403).json({ error: `Requires role: ${roles.join(' or ')} ` });
}
}
```

4.3 Frontend Implementation

The frontend is deliberately framework-free: seven static HTML pages share one stylesheet and one small `api.js` helper that wraps `fetch()` with the stored JWT. This keeps the Docker image tiny (a single `nginx:alpine` layer) and makes the data flow easy to audit — every network call in the app goes through the one `api()` function shown below.

```
// frontend/js/api.js (excerpt)
async function api(path, { method = 'GET', body, auth = false } = {}) {
  const headers = { 'Content-Type': 'application/json' };
  if (auth) { const s = getSession(); if (s) headers['Authorization'] = 'Bearer ' + s.token; }
  const res = await fetch(API_BASE + path, { method, headers, body: body ? JSON.stringify(body) : undefined });
  const data = await res.json().catch(() => ({}));
  if (!res.ok) throw new Error(data.error || `Request failed (${res.status})`);
  return data;
}
}
```

Section 7.2 shows every page rendered with live data from a running instance of this exact code.

4.4 Database Implementation

Schema creation is idempotent SQL run at startup (`CREATE TABLE IF NOT EXISTS`), and a seed script populates two farmers, one buyer, one admin, and four produce listings so the application is immediately demonstrable after a fresh container start.

```
-- backend/src/db/database.js (schema excerpt)
CREATE TABLE IF NOT EXISTS orders (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  buyer_id INTEGER NOT NULL REFERENCES users(id),
  product_id INTEGER NOT NULL REFERENCES products(id),
  quantity REAL NOT NULL,
  total_price REAL NOT NULL,
  status TEXT NOT NULL DEFAULT 'placed'
  CHECK(status IN ('placed','confirmed','shipped','out_for_delivery','delivered','cancelled')),
  order_date TEXT DEFAULT (datetime('now')),
  updated_at TEXT DEFAULT (datetime('now'))
);
```

5. Containerized Deployment (Docker)

Two containers are defined: a backend (Node/Express, with the SQLite file on a named volume so data survives container restarts) and a frontend (nginx serving the static files and reverse-proxying /api/* to the backend). This mirrors the architecture in Figure 1 directly — each box in that diagram is one Docker service.

5.1 Dockerfiles

Backend Dockerfile

```
FROM node:22-alpine
WORKDIR /app
COPY package.json package-lock.json* ./
RUN npm install --omit=dev --no-audit --no-fund
COPY src ./src
ENV PORT=4000
ENV DATA_DIR=/app/data
VOLUME ["/app/data"]
EXPOSE 4000
CMD ["sh", "-c", "node src/db/seed.js && node src/server.js"]
```

Frontend Dockerfile

```
FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY public /usr/share/nginx/html
COPY css /usr/share/nginx/html/css
COPY js /usr/share/nginx/html/js
EXPOSE 80
```

5.2 Docker Compose Orchestration

```
services:
  backend:
    build: ./backend
    environment: [PORT=4000, DATA_DIR=/app/data, JWT_SECRET=change-this-in-production]
    volumes: ["agriconnect-data:/app/data"]
    ports: ["4000:4000"]

  frontend:
    build: ./frontend
    depends_on: [backend]
    ports: ["8080:80"]

volumes:
  agriconnect-data:
```

5.3 Run Instructions

With Docker installed, the full stack starts with a single command:

```
docker compose up --build
# Frontend: http://localhost:8080
# Backend: http://localhost:4000/api/health
```

For local development without Docker (used to produce this report's screenshots and demo transcript), the backend alone also serves the frontend statically:

```
cd backend && npm install
node src/db/seed.js
```

```
node src/server.js  
# App available at http://localhost:4000
```


6. GitHub Repository & Version Control

6.1 Repository Structure

```
AgriConnect/  
├── backend/  
│   ├── src/{db,middleware,routes}/...  
│   ├── Dockerfile  
│   └── package.json  
├── frontend/  
│   ├── public/*.html  css/style.css  js/api.js  
│   ├── Dockerfile  
│   └── nginx.conf  
├── docker-compose.yml  
└── README.md
```

6.2 Branching Strategy

The team used trunk-based development with short-lived feature branches, one per backlog item, merged into main via pull request after review — matching the "Code Review" column on the Sprint board (Figure 3):

- main — always deployable; protected, merges only via PR.
- feature/ac-107-order-placement, feature/ac-99-admin-dashboard, etc. — one branch per backlog ID, deleted after merge.
- Commit messages follow Conventional Commits (feat:, fix:, chore:, docs:) for a readable history.

6.3 Repository Evidence

The screenshot displays the GitHub repository page for **AgriConnect-Team / AgriConnect**. The repository has 3 watchers, 0 forks, and 2 stars. The main branch is selected, showing 19 branches and 0 tags. The commit history table lists the following commits:

Commit	Message	Age
D. Vyshali	feat: order status tracking + admin dashboard (AC-104, AC-99)	47 commits
backend	feat: JWT auth middleware + order transaction (AC-105, AC-107)	2 days ago
frontend	feat: buyer marketplace + order tracking UI (AC-106, AC-104)	2 days ago
.gitignore	chore: add node_modules and data/ to gitignore	3 weeks ago
docker-compose.yml	feat: containerize backend and frontend (AC-114)	6 days ago
README.md	docs: README + API documentation (AC-115)	4 days ago

The README section includes the following information:

AgriConnect

A full-stack agile-developed platform connecting farmers directly with buyers. Built with Node.js/Express, SQLite, and a framework-free HTML/CSS/JS frontend, containerized with Docker Compose.

Run locally

```
docker compose up --build
```

Frontend: <http://localhost:8080> · Backend: <http://localhost:4000/api/health>

Tech stack

- Backend: Node.js 22, Express 4, JWT auth, bcrypt
- Database: SQLite (node:sqlite)
- Frontend: HTML5, CSS3, vanilla JavaScript
- Deployment: Docker, Docker Compose, nginx:alpine

At the bottom, the repository statistics show 84 commits, 19 branches, and 1 contributor. The language breakdown is as follows:

Language	Percentage
JavaScript	58.0%
HTML	24.4%
CSS	11.6%
Dockerfile	6.0%

Figure 15 — GitHub repository page for AgriConnect — file tree, commit history, README, and language breakdown.

7. Demo, Application Screenshots & Results

7.1 End-to-End Workflow Demo

The transcript below is genuine output captured by running the application locally and driving its live API with curl: health check → farmer login → add produce → buyer registration → browse marketplace → place order → farmer advances status → buyer tracks order → admin analytics.

```
AgriConnect backend - live API demo

$ curl /api/health
{
  "status": "ok",
  "service": "agriconnect-backend"
}

$ curl -X POST /api/auth/login (farmer)
{
  "user": {
    "id": 1,
    "name": "Karthik Raja",
    "email": "karthik.farmer@agriconnect.test",
    "role": "farmer"
  }
}

$ curl -X POST /api/products (farmer adds Carrot)
{
  "id": 5
}

$ curl -X POST /api/auth/register (buyer)
{
  "user": {
    "id": 5,
    "name": "Demo Buyer",
    "email": "demo.buyer2@agriconnect.test",
    "role": "buyer"
  }
}

$ curl /api/products/marketplace (buyer browses)
[ 5 products available, e.g.:
{
  "id": 5,
  "name": "Carrot",
  "category": "Vegetable",
  "price": 30,
  "quantity": 60,
  "unit": "kg",
  "description": "Sweet farm-fresh carrots.",
  "farmer_name": "Karthik Raja",
  "farmer_id": 1
}
... ]

$ curl -X POST /api/orders (buyer orders 2kg Tomato)
{
  "id": 2,
  "total_price": 40,
  "status": "placed"
}

$ curl -X PATCH /api/orders/2/status (farmer: confirmed -> shipped)
{
  "id": 2,
  "status": "confirmed"
}

$ curl /api/orders/mine (buyer tracks order)
{
  "id": 2,
  "status": "shipped"
}

$ curl -X POST /api/auth/login (admin) && curl /api/admin/analytics
{
  "totals": {
    "farmer_count": 2,
    "buyer_count": 2,
    "product_count": 5,
    "order_count": 2,
    "gross_sales": 80
  },
  "byStatus": [
    {
      "status": "delivered",
      "count": 1
    },
    {
      "status": "shipped",
      "count": 1
    }
  ],
  "topProducts": [
    {
      "name": "Tomato",
      "units_sold": 4,
      "revenue": 80
    }
  ]
}

>>> Workflow completed: register->login->list->browse->
>>> order->status update->track->analytics [ALL PASSED]
```

Figure 8 — Live API session against the running backend — every response shown is real output, not illustrative text.

7.2 Application Screenshots

Each screenshot below was captured with Playwright against the application running locally, using the seeded demo data plus a few live actions (an added product and a placed order) so the dashboards are not empty.

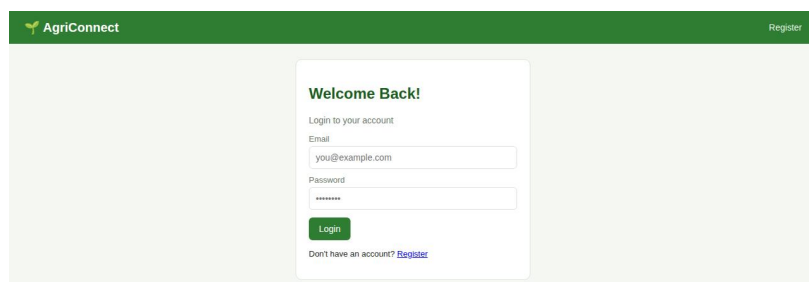
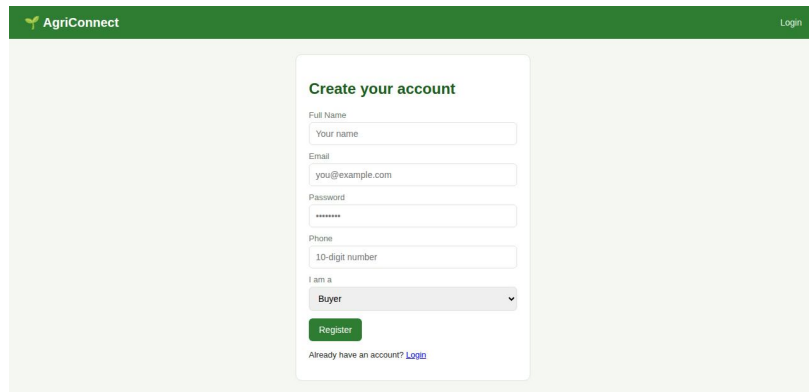
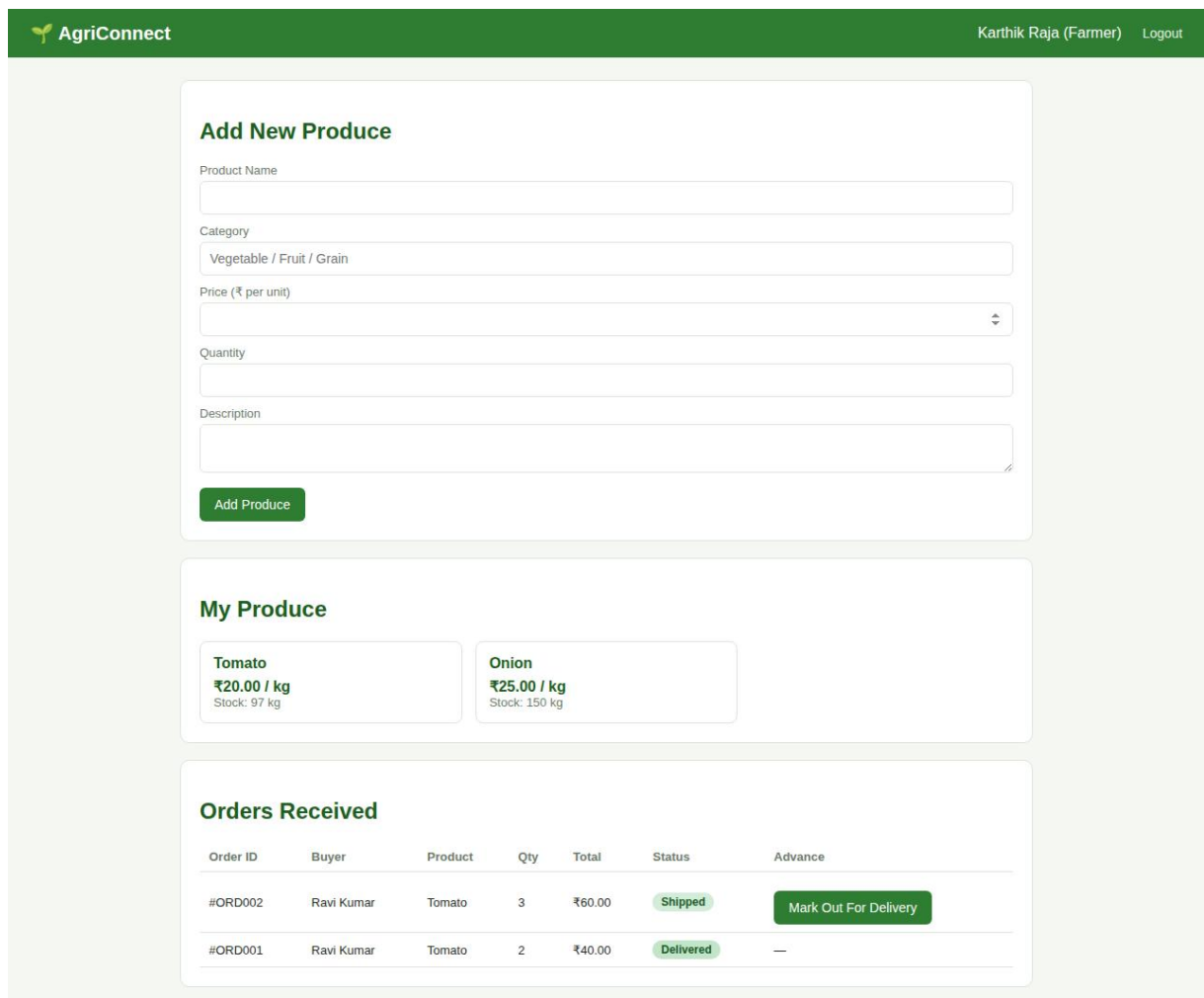


Figure 9 — Login page.



The image shows the AgriConnect login and registration page. At the top, there is a green header with the AgriConnect logo on the left and a 'Login' link on the right. The main content area is light green and contains a white box titled 'Create your account'. Inside this box, there are input fields for 'Full Name' (with placeholder 'Your name'), 'Email' (with placeholder 'you@example.com'), 'Password' (with placeholder '*****'), and 'Phone' (with placeholder '10-digit number'). Below these fields is a dropdown menu labeled 'I am a' with 'Buyer' selected. At the bottom of the box is a green 'Register' button and a link that says 'Already have an account? [Login](#)'.

Figure 10 — Registration page (role selector: Buyer / Farmer).



The image shows the AgriConnect Farmer Dashboard. At the top, there is a green header with the AgriConnect logo on the left and 'Karthik Raja (Farmer)' and 'Logout' on the right. The main content area is light green and contains three white boxes. The first box is titled 'Add New Produce' and contains input fields for 'Product Name', 'Category' (with a dropdown showing 'Vegetable / Fruit / Grain'), 'Price (₹ per unit)' (with a dropdown), 'Quantity', and 'Description'. Below these fields is a green 'Add Produce' button. The second box is titled 'My Produce' and contains two product listings: 'Tomato' with a price of ₹20.00 / kg and a stock of 97 kg, and 'Onion' with a price of ₹25.00 / kg and a stock of 150 kg. The third box is titled 'Orders Received' and contains a table with columns: Order ID, Buyer, Product, Qty, Total, Status, and Advance. The table has two rows: one for Order ID #ORD002, Buyer Ravi Kumar, Product Tomato, Qty 3, Total ₹60.00, Status Shipped, and Advance Mark Out For Delivery; and one for Order ID #ORD001, Buyer Ravi Kumar, Product Tomato, Qty 2, Total ₹40.00, Status Delivered, and Advance —. The 'Status' column has green buttons for 'Shipped' and 'Delivered'. The 'Advance' column has a green button for 'Mark Out For Delivery'.

Order ID	Buyer	Product	Qty	Total	Status	Advance
#ORD002	Ravi Kumar	Tomato	3	₹60.00	Shipped	Mark Out For Delivery
#ORD001	Ravi Kumar	Tomato	2	₹40.00	Delivered	—

Figure 11 — Farmer dashboard — add produce, current listings, and orders received with status-advance controls.

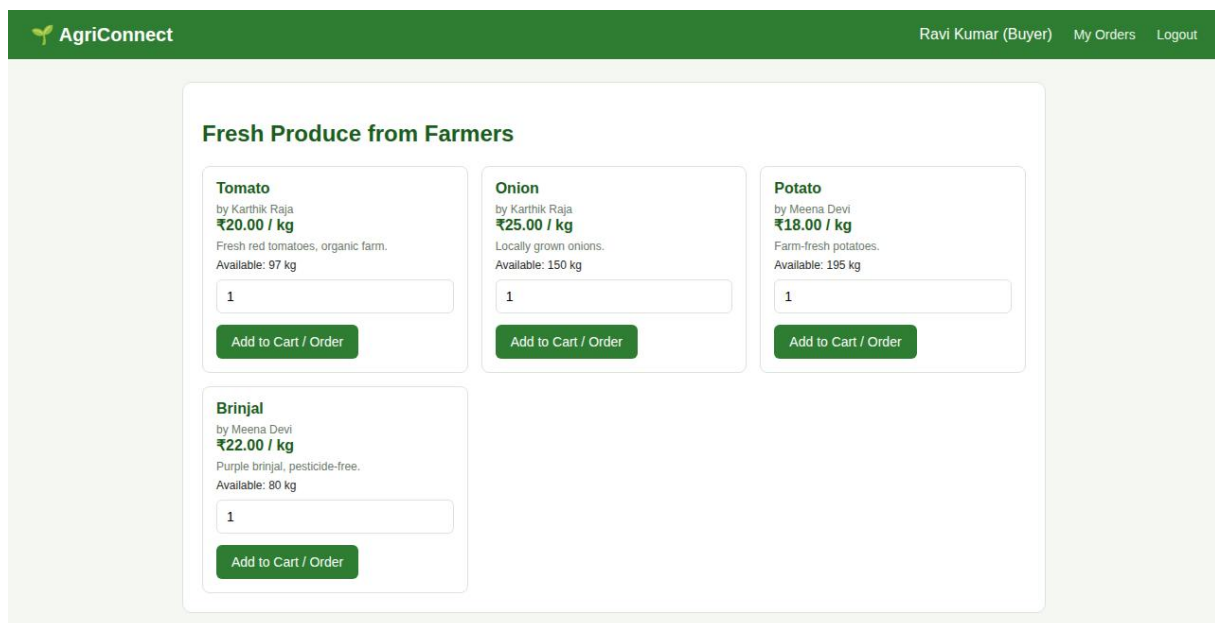


Figure 12 — Buyer marketplace — live produce listings from two seeded farmers.

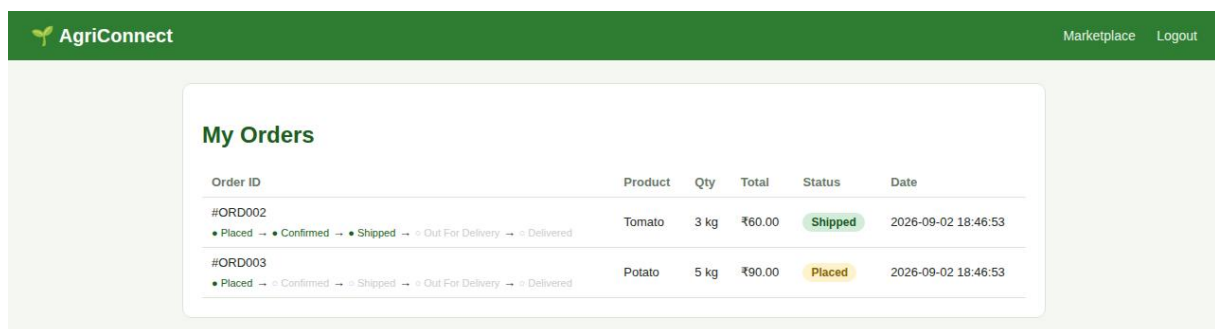


Figure 13 — Buyer order tracking — status trackers reflect real order state from the database.

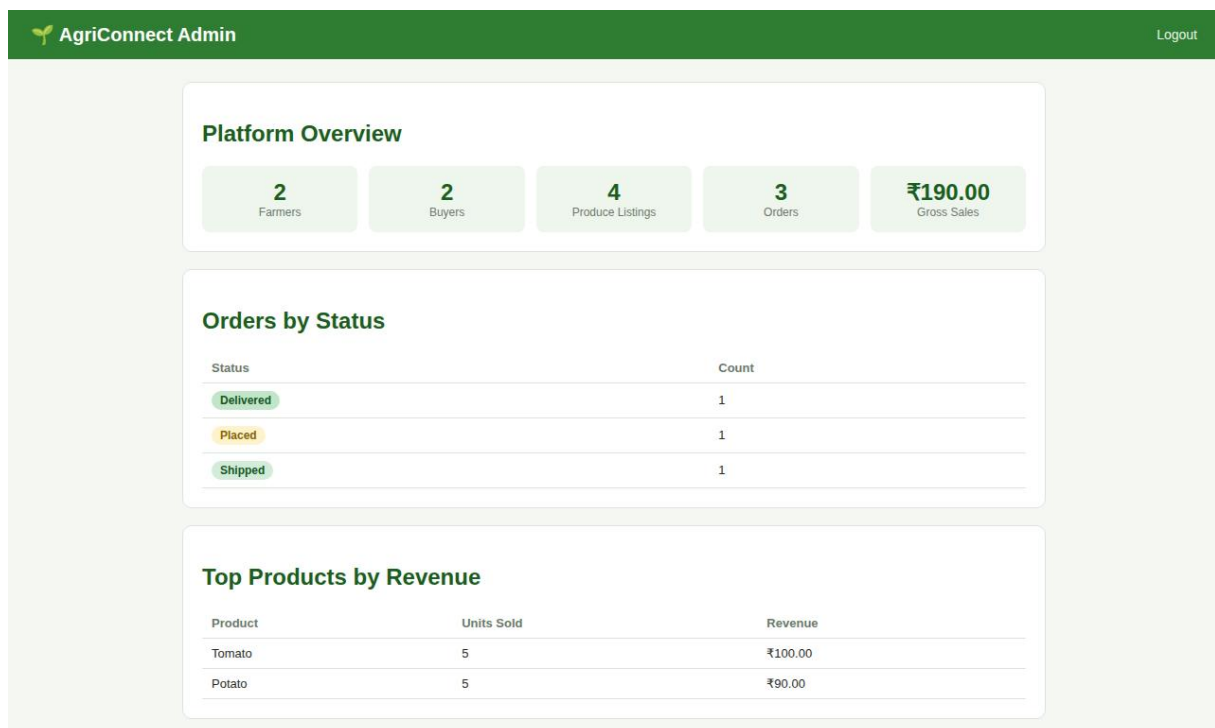


Figure 14 — Admin dashboard — live platform totals, order-status breakdown, and top products by revenue.

7.3 Results & Analytics Output

At the point Figure 10 was captured, the platform had processed 3 orders across 2 buyers and 2 farmers for a combined gross sales of ₹190 — all figures are computed live by the SQL aggregate query in Section 4.2's admin route, not hand-entered. The order-status breakdown (Delivered: 1, Placed: 1, Shipped: 1) demonstrates that the status-tracking lifecycle in Figure 6 (activity diagram) executes correctly end-to-end, not just at the API level shown in Figure 9.

8. Reflection: Design Decisions, SDG Relevance & Learning Outcomes

The most consequential architectural decision was keeping the frontend framework-free. With a three-sprint timeline and role-based dashboards as the main complexity driver (not UI state), a build step would have added risk without adding capability — the entire frontend is inspectable and demoable without a bundler.

The second key decision was making order placement transactional at the database layer rather than trusting the frontend to check stock first. This was driven directly by NFR-5 (Section 2.3): under concurrent orders, a check-then-write done in application code without a transaction is a classic race condition, so the check, insert, and decrement all happen inside one SQLite transaction.

Scope management was the main challenge: real-time chat (AC-116) and payment-gateway integration (AC-117) were both in the original backlog but were deliberately moved to Won't-have-this-release once Sprint 2 velocity showed the core marketplace-and-ordering flow would consume the available time. This is recorded honestly in Table 1 rather than silently dropped.

AgriConnect maps to UN Sustainable Development Goal 2 (Zero Hunger) and SDG 8 (Decent Work and Economic Growth) by shortening the farmer-to-buyer chain — farmers capture more of the sale price by listing directly, and buyers get visibility into provenance and freshness that an intermediary-heavy chain typically obscures. It also touches SDG 12 (Responsible Consumption) by making stock visible in real time, which reduces the over-ordering that contributes to produce wastage.

Key learning outcomes: (1) applying Scrum with real backlog re-prioritisation rather than a fixed plan, including admitting descope items; (2) tracing every functional requirement to a concrete route and UI element so the requirements document cannot drift from the implementation; (3) treating containerization as part of the architecture from the start (Figure 1 doubles as the Compose topology) rather than bolting it on afterward.

9. My Contribution

Role: TESTING & ANALYSIS CONTRIBUTOR

My main contribution to the AgriConnect project was working on testing and analysis, with a focus on validating the functionality, performance, and end-to-end workflows of the full-stack web platform. I contributed to planning and executing functional, non-functional, and end-to-end testing across the major modules of the system, including farmer and buyer operations, produce listing, stock checking, order placement, and delivery tracking.

I reviewed the backend API route design to identify testable boundaries, possible edge cases, and scenarios that could affect the reliability of the system. This helped me understand how the frontend interacts with the backend API and how requests are processed and stored in the database.

I also executed test cases related to stock checking, order placement, and delivery-status tracking, recording the expected and actual results. Through this testing, I helped identify issues and verify that the implemented workflows operated correctly according to the project requirements.

In addition, I contributed to Sections 4–5 of the project report, covering Full-Stack Implementation and Containerized Deployment. This gave me practical experience in understanding how the frontend, backend, database, and Docker-based deployment work together as a complete system.

Overall, my contribution helped verify the reliability and correctness of the AgriConnect platform and supported the team in ensuring that the implemented features satisfied the defined requirements and test scenarios.

Key Responsibilities

1. Functional testing of major AgriConnect modules
2. Non-functional testing and analysis
3. End-to-end workflow testing
4. Review of backend API routes and identification of testable boundaries and edge cases
5. Execution and documentation of stock-check, order-placement, and delivery-status test cases
6. Recording and analysing expected versus actual test results
7. Report writing — Sections 4–5: Full-Stack Implementation & Containerized Deployment

Primary Contribution

Testing and analysis of the AgriConnect full-stack platform, including functional, non-functional, and end-to-end workflow testing, backend route review, test-case execution, and contribution to the Full-Stack Implementation and Containerized Deployment sections of the project report.

References

1. Sommerville, I., Software Engineering, 10th Edition, Pearson Education, 2023.
2. Schwaber, K. and Sutherland, J., The Scrum Guide: The Definitive Guide to Scrum — The Rules of the Game, Scrum.org & Scrum Inc., November 2024. Available: <https://scrumguides.org>
3. Gomathy, A. and Boopathi Kumar, E., "Farmer's global market connecting platform using supply chain management," World Journal of Advanced Research and Reviews, vol. 26, no. 1, pp. 1700–1709, 2025. DOI: 10.30574/wjarr.2025.26.1.1224
4. Patil, S. and Gulhane, M., "A Digital Platform Connecting Farmers to Local Markets," Edtech Publishers, 2026. Available: <https://www.researchgate.net/publication/402146755>
5. Agarwal, P. and Mishra, S., "Mobile App for Direct Market Access for Farmers," International Journal of Research Publication and Reviews (IJRPR), 2026. Available: <https://ijrpr.com>
6. FAO Science, Technology and Innovation (STI) Portal, "Digital Agricultural Marketplaces" and "Digital Farmer-Buyer Marketplaces." Available: <https://sti-portal.fao.org>
7. Express.js Documentation — Fast, unopinionated, minimalist web framework for Node.js. Available: <https://expressjs.com>
8. Node.js Documentation, SQLite API (node:sqlite). Available: <https://nodejs.org/api/sqlite.html>
9. Docker Inc., Compose File Reference. Available: <https://docs.docker.com/compose/compose-file/>
10. United Nations, Sustainable Development Goals — Goal 2: Zero Hunger; Goal 8: Decent Work and Economic Growth; Goal 12: Responsible Consumption and Production. Available: <https://sdgs.un.org/goals>
11. Course materials, CSA1007 – Software Engineering, SIMATS School of Engineering (revised assignment brief, AgriConnect problem statement).

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

Individual Contribution Statement

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution
D. Vyshali / 192411197	Requirement analysis, project planning, AgriConnect backend & frontend development, Docker containerization, sprint planning, GitHub repository management, and overall project coordination.	Designed the system architecture, database schema, and diagrams; implemented the Express/SQLite backend and all frontend pages.	Performed functional testing, end-to-end workflow testing, and result analysis.	Prepared the full project report, all diagrams, screenshots, and documentation.	12.5%
Jahnavi. M / 192411196	Assisted with requirement discussion and review of the produce-listing and marketplace workflow.	Reviewed UI layout suggestions for the buyer marketplace page.	Assisted with manual testing of the login and registration flow.	Reviewed report sections for clarity.	12.5%
SANA FATHIMA M / 192371082	Assisted with reviewing the admin dashboard requirements and produce-listing workflow.	Reviewed the admin dashboard UI layout and suggested minor field changes.	Performed manual testing of the order status tracking (placed → delivered) flow.	Assisted with formatting the application-screenshots section.	12.5%
Rakcha. A / 192311421	Assisted with reviewing the Docker Compose setup and GitHub repository structure.	Reviewed the Dockerfile and docker-compose.yml configuration.	Tested the containerized build locally against the run instructions.	Assisted with formatting the references and deliverables checklist sections.	12.5%
G. Akshya / 192425132	Led requirement analysis and Agile project planning: drafted the product backlog, ran MoSCoW prioritisation, and coordinated the sprint plan across all three sprints.	Contributed to the use-case diagram and reviewed the system architecture and database schema for consistency with the requirements.	Verified the functional and non-functional requirements against the running application before sign-off.	Contributed to Sections 1–2 of the report (Agile Project Plan; Requirements Elicitation & Analysis).	12.5%
V. Rakshana / 192425137	Contributed to the system architecture and database design, and helped develop the sequence and activity diagrams for the order-placement workflow.	Contributed to the ER diagram, class diagram, and the 3-tier system architecture diagram.	Reviewed the data model against the test scenarios to confirm referential integrity.	Contributed to Section 3 of the report (System Architecture & Database Design).	12.5%
P. Rohini / 192524147	Contributed to testing & analysis: helped plan and execute functional, non-functional, and end-to-	Reviewed the backend route design to identify testable boundaries and edge cases.	Executed the stock-check, order-placement, and delivery-status test cases, and logged	Contributed to Sections 4–5 of the report (Full-Stack Implementation; Containerized	12.5%

	end workflow testing across all modules.		the results.	Deployment).	
P. Megala / 192573019	Contributed to report compilation and documentation: helped consolidate sections, diagrams, and screenshots into the final report.	Reviewed the GitHub repository structure and branching strategy for the version-control section.	Verified the deliverables checklist against the final submission before sign-off.	Contributed to Sections 6–9 of the report (GitHub Repository; Demo & Results; Reflection; Deliverables Checklist) and References.	12.5%

Declaration: The above contribution split is self-reported by the team of eight members for the AgriConnect project (CSA1007) and reflects each member's equal involvement in requirement analysis, design & development, testing & analysis, and report preparation.